UAV

Challenge 2

Step 0: setup

```
pvn@pvn-VMware-Virtual-Platform:~$ mkdir ~/uavh2 && cd ~/uavh2
pvn@pvn-VMware-Virtual-Platform:~/uavh2$ script -a lab_log.txt
Script started, output log file is 'lab_log.txt'.
pvn@pvn-VMware-Virtual-Platform:~/uavh2$
```

```
pvn@pvn-VMware-Virtual-Platform:~/uavh2$ python3 -m venv UAV2
pvn@pvn-VMware-Virtual-Platform:~/uavh2$ source UAV2/bin/activate
(UAV2) pvn@pvn-VMware-Virtual-Platform:~/uavh2$
```

Step 1: shell inspection

```
pvn@pvn-VMware-Virtual-Platform:~/uavh2$ head -5 detections.csv
t_s,channel
0.01937,78
0.04316,78
0.19903,61
0.21768,13
pvn@pvn-VMware-Virtual-Platform:~/uavh2$ wc -l detections.csv
420 detections.csv
pvn@pvn-VMware-Virtual-Platform:~/uavh2$ tr ',' '\n' < watchlist.txt | wc -l
32
```

channel frequency in the log

```
pvn@pvn-VMware-Virtual-Platform:~/uavh2$ tail -n +2 detections.csv | cut -d, -f2
 | sort -n | uniq -c | sort -rn | head
     26 38
     19 32
     18 21
     17 73
     17 65
     17 63
     17 51
     16 78
     16 55
     15 61
```

Your outputs match what I expected:

- **420 lines** is 419 detections plus the header, which agrees with the other AI's count.
- **32 watchlist channels** confirms the receiver covered only 32 of 80.

- **Channel counts:** with 419 detections over 32 channels, the average is about 13 per channel. Channel 38 at 26 is high, but a log mixing the real UAV with another detachment won't be uniform, so don't read anything into it yet. Just note it.

One check still open

The README says a hop onto an unwatched channel produces no entry. Confirm that every channel in the log is on the watchlist:



TO Find:

1. **How big is the gap between the top score and the second?**
2. **What is the median score? A random state should match about 1 in 80, which is about 5 of 419.**
3. **What fraction of 419 does the winner reproduce?**

Sep1_hops.py. Write the step() and channel_of() functions from the README and print the first 10 channels for a test state of your choice. This checks that you understand the generator before attacking it.

#output:

```
(UAV2) pvn@pvn-VMware-Virtual-Platform:~/uavh2$ python3 hops.py
0 0x1 1
1 0x2 2
2 0x4 4
3 0x8 8
4 0x10 16
5 0x20 32
6 0x40 64
7 0x80 1
8 0x100 2
9 0x200 4
```

Output decision:

1. hops.py: what the output proves

Take the test state 0x1. The filter splits the 20 bits into three pieces:

- t1 = the low 7 bits
- t2 = the next 7 bits
- t3 = the next 6 bits

At hop 7 the state is 0x80. The set bit has moved from t1 into t2. Now t1 = 0, t2 = 1, t3 = 0, so the channel is (0 ^ 1) + 0 = 1. The output wraps back to 1, which confirms me understand the bit layout.

Step 3: step2_bruteforce.py. This is your task, and I'll leave the structure to you:

- Convert timestamps to hop numbers.
- For each s0 from 1 to $2^{20}-1$, run the generator up to the last hop and count matches.
- Print the top 5 scores.

```
(UAV2) pvn@pvn-VMware-Virtual-Platform:~/uavh2$ python3 bruteforce.py
detections: 419  last hop: 998
0x9e3b1 355
0x9f5db 19
0x7678e 19
0x97b1d 19
0xff739 19
median score: 5.0
```

Step A: time to hop. round(t / 0.02) turns a timestamp like 0.19903 into hop 10. The README says the grid is exactly 20 ms with a little jitter. Rounding absorbs the jitter.

Step B: all candidates at once. s = np.arange(1, 1<<20) is an array of 1,048,575 states, one for every possible secret. Because the secret is only 20 bits, we can try

them all. The all-zero state is skipped because an XOR LFSR would stay at zero forever.

Step C: stepping them together. Each loop iteration applies step() to the entire array, which advances every candidate by one hop. At hops where the log has a detection, we compute each candidate's channel and add 1 to its score where it matches the observed channel. This is much faster than a Python loop of 1M states × ~1000 steps.

Step D: ranking. argsort sorts candidates by score.

Step 4: step3_predict.py. Take the winning state, step it forward, and print channels 1000–1011.

Output:



Decisions for your document

1. **Method:** brute force over all $2^{20}-1$ states, scored by match count.
2. **Why brute force:** the unknown is only 20 bits, and the nonlinear filter blocks reading the state directly from the channels.
3. **Confidence:** 355/419 against a runner-up of 19 and a median of 5.
4. **Unmatched rows:** these come from the other detachment's contamination.
5. **Files to include:** hops.py (understanding), bruteforce.py (the key recovery), step3_predict.py (the final answer), plus the shell checks and comm output.
6. **Mistake to avoid:** I found 0x9e3b1, not the 0x3C762 the other AI gave. Use your own value.

#hops.py

```
MASK = 0xFFFFF


def step(s):
    fb = ((s >> 19) ^ (s >> 16)) & 1
```

```
    return ((s << 1) | fb) & MASK


def channel_of(s):
    t1 = s & 0x7F
    t2 = (s >> 7) & 0x7F
    t3 = (s >> 14) & 0x3F
    return ((t1 ^ t2) + (t2 * t3)) % 80

s = 0x00001                    # arbitrary test state
for k in range(10):
    print(k, hex(s), channel_of(s))
    s = step(s)



#bruteforce.py
import csv
import numpy as np

MASK = 0xFFFFF

# 1. timestamps -> hop numbers
det = []
for r in csv.DictReader(open("detections.csv")):
    det.append((round(float(r["t_s"]) / 0.02), int(r["channel"])))
last_hop = max(h for h, c in det)
print("detections:", len(det), " last hop:", last_hop)

# group by hop (two detections can fall in the same hop)
```

```
by_hop = {}
for h, c in det:
    by_hop.setdefault(h, []).append(c)

# 2. every possible non-zero initial state, as one array
s = np.arange(1, 1 << 20, dtype=np.int64)
s0 = s.copy()
score = np.zeros(len(s), dtype=np.int32)

# 3. run all candidates forward together, scoring at hops that have detections
for k in range(last_hop + 1):
    if k in by_hop:
        t1 = s & 0x7F
        t2 = (s >> 7) & 0x7F
        t3 = (s >> 14) & 0x3F
        ch = ((t1 ^ t2) + t2 * t3) % 80
        for observed in by_hop[k]:
            score += (ch == observed)
    fb = ((s >> 19) ^ (s >> 16)) & 1
    s = ((s << 1) | fb) & MASK



# 4. report the best candidates
order = np.argsort(-score)[:5]
for i in order:
    print(hex(s0[i]), score[i])
print("median score:", np.median(score))
```

```
#predict.py
MASK = 0xFFFFF

def step(s):
    fb = ((s >> 19) ^ (s >> 16)) & 1
    return ((s << 1) | fb) & MASK

def channel_of(s):
    t1 = s & 0x7F
    t2 = (s >> 7) & 0x7F
    t3 = (s >> 14) & 0x3F
    return ((t1 ^ t2) + (t2 * t3)) % 80

s = 0x9E3B1
out = []
for k in range(1012):
    if k >= 1000:
        out.append(channel_of(s))
    s = step(s)

print("KST{" + "-".join(map(str, out)) + "}")
```